Documentation technique — Zoo d'Arcadia

1. Réflexions techniques initiales

Le site existait d'abord sous forme de pages HTML statiques. Une page HTML ne retient et ne calcule rien : elle reste identique quel que soit le visiteur. Or le cahier des charges demandait un back-office à trois rôles, la gestion des habitats, des animaux et des services par l'administrateur, la modération des avis, le suivi de l'alimentation et de la santé des animaux, et un compteur de consultations. Il fallait donc un langage exécuté côté serveur, simple à mettre en place et maintenir.

PHP, sans framework. PHP gère l'authentification, les rôles et la persistance des données. J'ai écarté les frameworks comme Symfony ou Laravel. Ils fournissent clés en main un routeur, une couche d'accès aux données et des protections de sécurité. Mais ils masquent aussi les mécanismes que ce projet doit démontrer : la requête préparée, l'échappement en sortie et la gestion de session. En les écrivant moi-même, je peux les expliquer ligne par ligne. L'énoncé cite d'ailleurs « PHP avec utilisation de PDO » comme exemple de stack.

MariaDB pour les données relationnelles. J'utilise MariaDB en local et en production. C'est un dérivé compatible de MySQL, et c'est le SGBD que fournit XAMPP. Pour les besoins du projet, c'était adapté : il s'agit de tables régulières, de clés étrangères et d'agrégats simples. Rester sur MariaDB évitait une installation supplémentaire, sans rien perdre en fonctionnalités.

Bootstrap 5 pour l'interface. Je l'ai préféré à un CSS entièrement manuel. Sa grille responsive couvre les six maquettes attendues sans réécrire de points de rupture. Ses composants interactifs (menu dépliant, onglets, modale) sont utilisables au clavier et correctement annoncés aux lecteurs d'écran, ce qui est long à obtenir en manuel. Enfin, il se charge depuis un CDN, sans étape de compilation. La charte graphique du zoo est appliquée par-dessus, dans `style.css`, pilotée par des variables CSS.

JavaScript sans bibliothèque. Le site compte une centaine de lignes de JavaScript : l'année du pied de page et la fiche animal asynchrone. Il est donc suffisant sans ajout de bibliothèque.

MongoDB pour le compteur de consultations. L'US 11 exige de stocker le nombre de consultations de chaque fiche animal dans une base non relationnelle. Ce besoin est précis et isolé : beaucoup d'écritures, aucune relation avec les autres données,

pas de schéma figé. C'est un cas d'usage typique du NoSQL. Les données structurées du projet (utilisateurs, animaux, habitats, avis, services) restent dans MariaDB. J'ai donc choisi MongoDB, qui est accessible en tant que débutant et bien documenté.

2. Configuration de l'environnement de travail

Serveur local. Le développement s'est fait sous XAMPP, qui fournit Apache, PHP 8.2 et MariaDB 10.4. La production utilise des versions plus récentes, PHP 8.4.25 et MariaDB 11.4. J'ai donc vérifié la compatibilité avant le déploiement (section 6).

Environnement conteneurisé. En complément de XAMPP, le projet peut être lancé dans des conteneurs Docker. Un conteneur est un environnement d'exécution isolé, décrit par un fichier texte : il embarque le système minimal, le serveur et les versions exactes dont l'application a besoin. Il ne s'agit pas d'une machine virtuelle — rien n'est émulé, les conteneurs partagent le noyau du système hôte — mais le résultat est comparable : l'application tourne toujours dans le même environnement, quelle que soit la machine.

Trois conteneurs sont décrits dans `compose.yaml`, un par composant :

Conteneur	Image	Rôle
`web`	`php:8.4-apache`	Apache et PHP, dans la version de production
`mariadb`	`mariadb:11.4`	Base relationnelle
`mongo`	`mongo:8.0`	Compteur de consultations (US 11)

Deux fichiers suffisent à décrire l'ensemble. Le `Dockerfile` construit l'image du site : il part de l'image officielle PHP, y installe l'extension `pdo_mysql`, compile l'extension MongoDB, puis active `mod_rewrite` et `AllowOverride` pour qu'Apache tienne compte du fichier `.htaccess` du projet. Le `compose.yaml` décrit les trois services, leurs variables d'environnement, leurs volumes et leurs dépendances.

Les mots de passe des conteneurs vivent dans un fichier `.env`, exclu du dépôt, dont le modèle `.env.example` est versionné — même principe que `config/config.php` et `config/config.exemple.php`.

Conséquence sur le code applicatif : une ligne par connexion. Un fichier `includes/configuration.php` lit désormais les paramètres dans les variables d'environnement, avec `config/config.php` en valeur de repli. Le même code s'exécute donc sous XAMPP, dans un conteneur et chez l'hébergeur, sans aucune condition d'environnement écrite dans l'application. L'ordre de priorité est important : les variables d'environnement l'emportent sur le fichier, car le dossier du projet est

monté dans le conteneur et son `config.php` local y est visible.

Lancement : `docker compose up --build`, puis `http://localhost:8080`. Les tables sont créées et peuplées automatiquement, MariaDB exécutant `sql/01_schema.sql` puis `sql/02_donnees.sql` au premier démarrage.

La production n'utilise pas de conteneurs. Le site déployé tourne sur un hébergement mutualisé classique, dont l'offre gratuite n'exécute pas de conteneurs. Docker sert ici d'environnement de développement et de démonstration.

Base NoSQL. MongoDB Community Server tourne en local sur son port par défaut, 27017. PHP y accède par l'extension `mongodb`. Pour l'installer sous Windows, j'ai déposé dans le dossier `ext/` de XAMPP la DLL correspondant exactement à ma version de PHP, puis déclaré `extension=php_mongodb.dll` dans `php.ini`. MongoDB Compass m'a servi à inspecter les documents. C'est un client graphique, pas un serveur. En production, le serveur local est remplacé par un cluster MongoDB Atlas : seule la chaîne de connexion change, le code reste identique.

Configuration hors du code. Les identifiants de connexion aux deux bases sont rangés dans `config/config.php`, un fichier exclu du dépôt par `.gitignore`. Le dépôt contient à sa place `config/config.exemple.php`, qui en donne la structure sans aucune valeur réelle. Pour installer le projet sur une nouvelle machine, il suffit de copier ce modèle et de le remplir. Le `README.md` décrit cette étape.

Éditeur. J'utilise VS Code, avec Prettier pour l'indentation et la mise en forme, et Quokka pour tester un morceau de JavaScript sans passer par le navigateur. L'extension Live Server a servi pendant la phase statique uniquement. Elle sert des fichiers mais n'exécute pas PHP : depuis le passage à PHP, je consulte les pages via Apache, sur `localhost`.

Versionnement. J'utilise Git, avec un dépôt public sur GitHub. Les branches suivent le modèle demandé par l'énoncé :

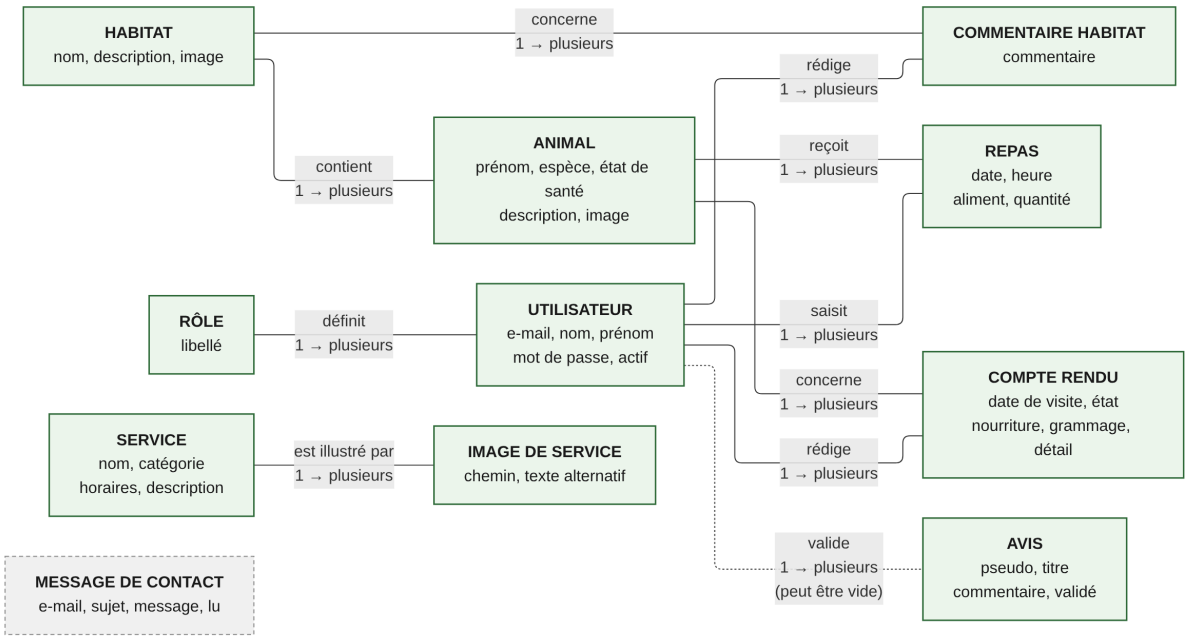
- `main` reflète ce qui tourne en production ;
- `develop` reçoit les fonctionnalités terminées ;
- chaque fonctionnalité a sa propre branche, issue de `develop` et fusionnée après test.

Les fusions sont faites en `--no-ff`, pour que chaque fonctionnalité reste identifiable dans l'historique. La documentation de gestion de projet détaille cette méthode.

3. Modèle conceptuel de données (MCD)

Le schéma relationnel compte 11 tables. Le MCD ci-dessous en reprend les 11 entités et leurs relations. Les clés, les types et les contraintes figurent dans `sql/01_schema.sql`.

Chaque lien porte un verbe et une cardinalité écrite en toutes lettres. Il se lit de gauche à droite : « HABITAT — contient, 1 → plusieurs — ANIMAL » signifie qu'un habitat contient plusieurs animaux, et qu'un animal appartient à un seul habitat. J'ai écrit les cardinalités en clair pour que le schéma reste lisible par quelqu'un qui n'est pas informaticien.



Un avis peut n'avoir aucun validateur. Le lien vers AVIS est en pointillés. Quand un visiteur dépose un avis, aucun employé ne l'a encore validé : la colonne `valide_par` reste à `NULL` jusqu'à la validation.

MESSAGE DE CONTACT n'a volontairement aucune relation. Un message de contact est déposé par un visiteur anonyme. Le rattacher à un compte n'aurait aucun sens, puisqu'aucun visiteur n'en possède.

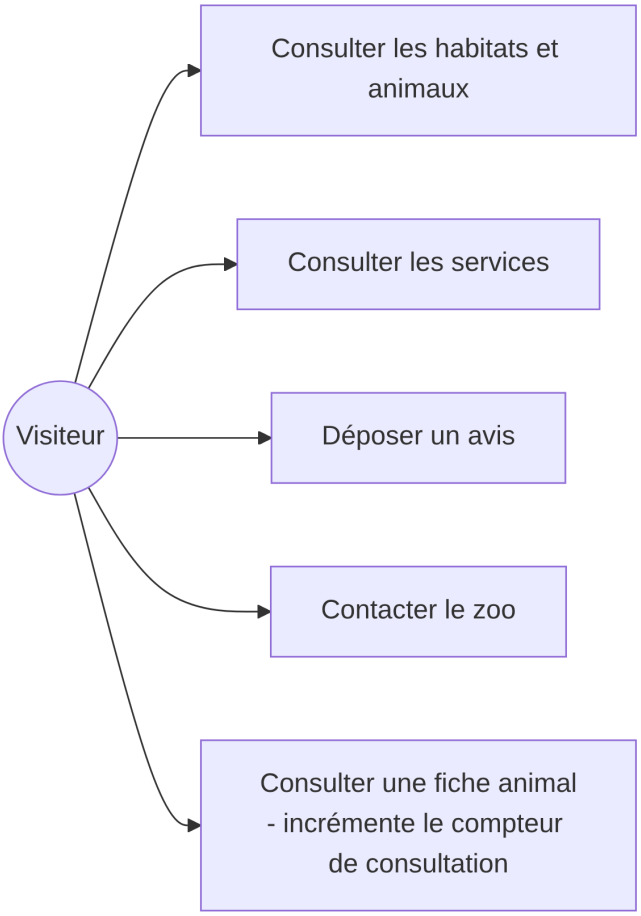
Les images des animaux et des habitats sont des attributs, pas une entité. Seuls les services ont une entité dédiée, IMAGE DE SERVICE, parce qu'eux seuls affichent plusieurs photos. Un animal et un habitat n'en portent qu'une : une entité séparée aurait ajouté une jointure sans bénéfice. Si le besoin évoluait vers une galerie, la structure d'IMAGE DE SERVICE serait reproduite à l'identique.

La date et l'heure du repas sont deux attributs distincts. L'US 7 demande la date et l'heure. Dans la base, ce sont deux colonnes (`date_repas`, `heure_repas`) plutôt qu'un unique `DATETIME`. Pour regrouper les repas par journée, je compare directement la date, sans fonction de conversion dans chaque requête.

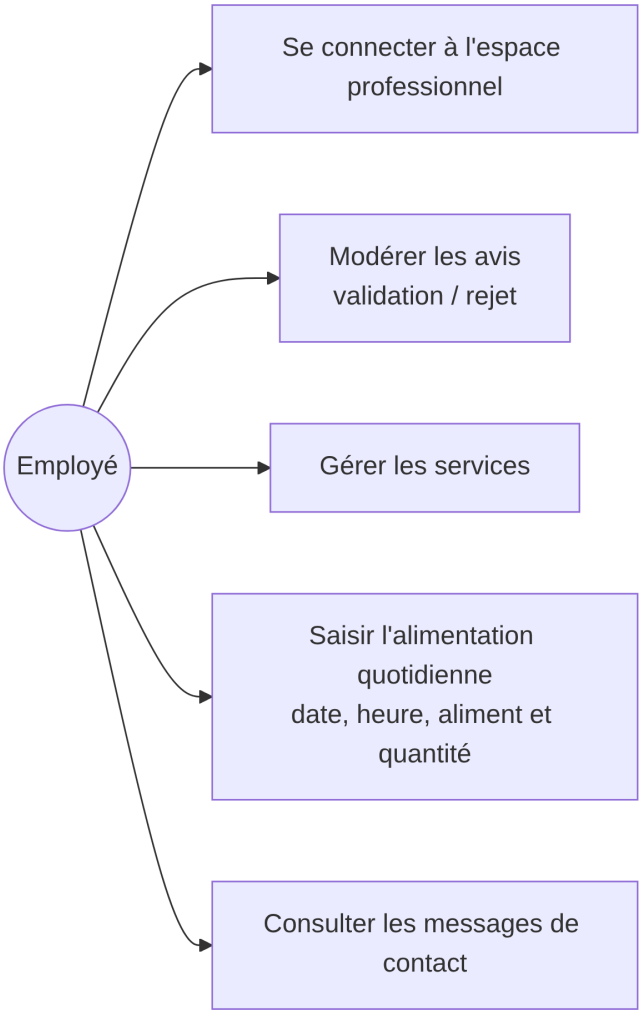
Le compteur de consultations vit à part. Il est stocké dans une collection MongoDB, sans lien référentiel avec les tables MariaDB. Chaque document associe un identifiant d'animal à un nombre de vues : `{ animal_id, views }`. Le code PHP interroge les deux bases séparément et assemble le résultat au moment de l'affichage, par exemple pour classer les animaux les plus consultés dans le tableau de bord administrateur. L'absence de lien entre les deux bases est une limite assumée, déjà documentée dans le dossier de projet.

4. Diagrammes de cas d'utilisation par rôle

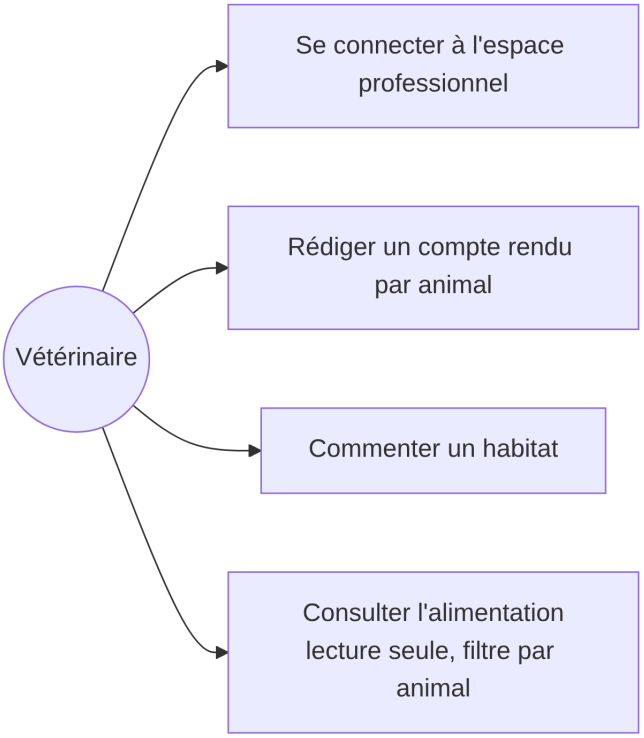
Visiteur



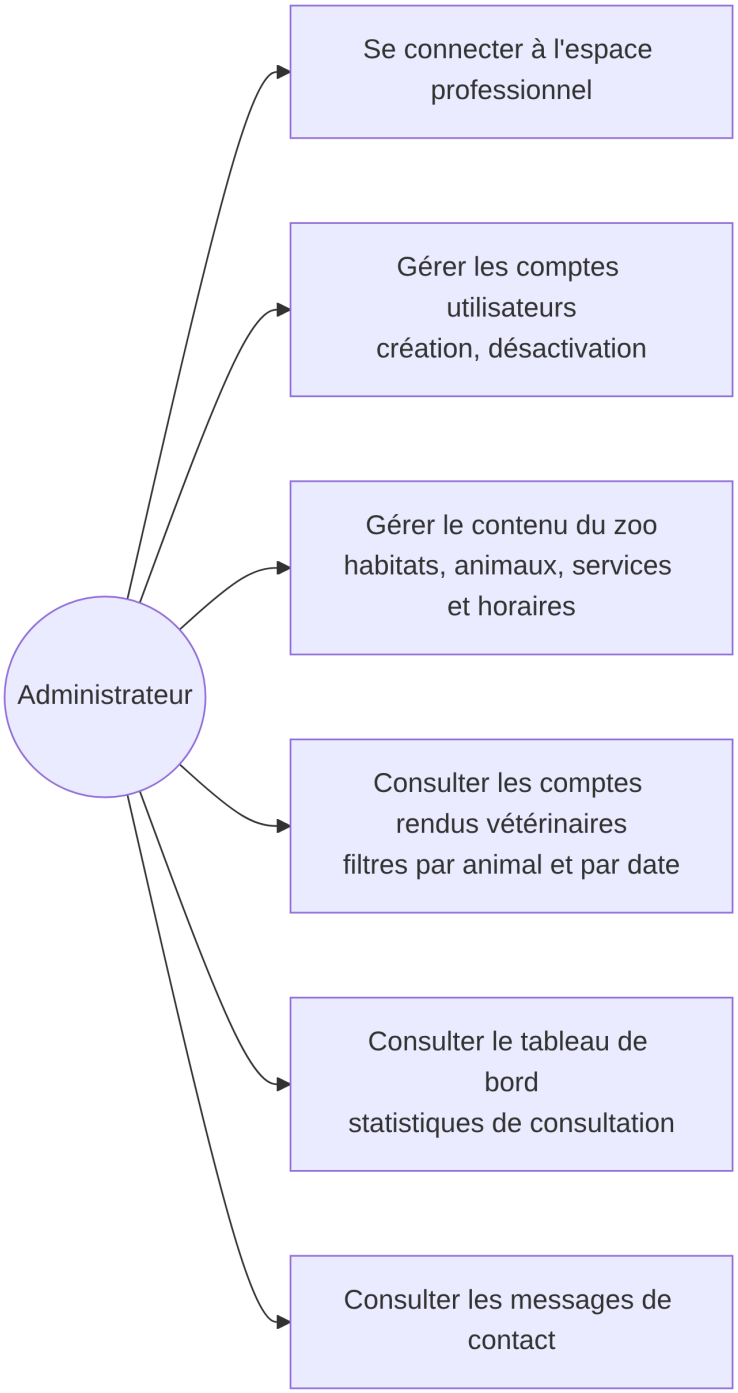
Employé



Vétérinaire

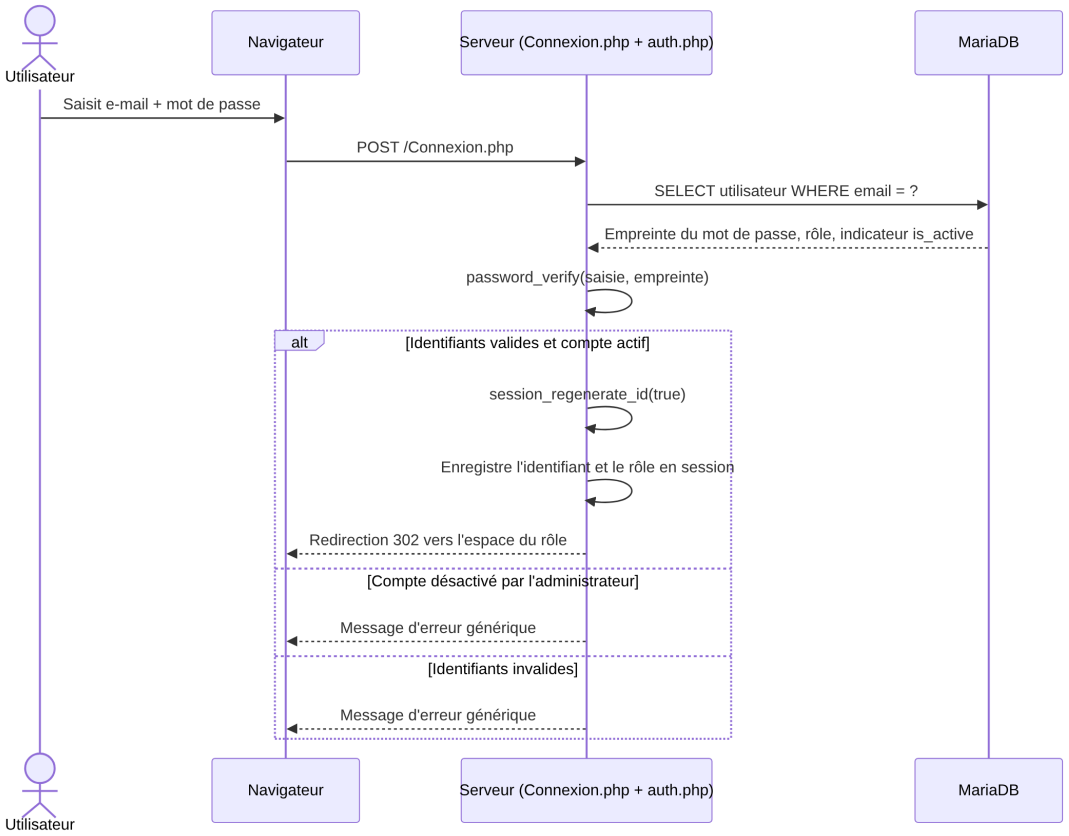


Administrateur



5. Diagrammes de séquence

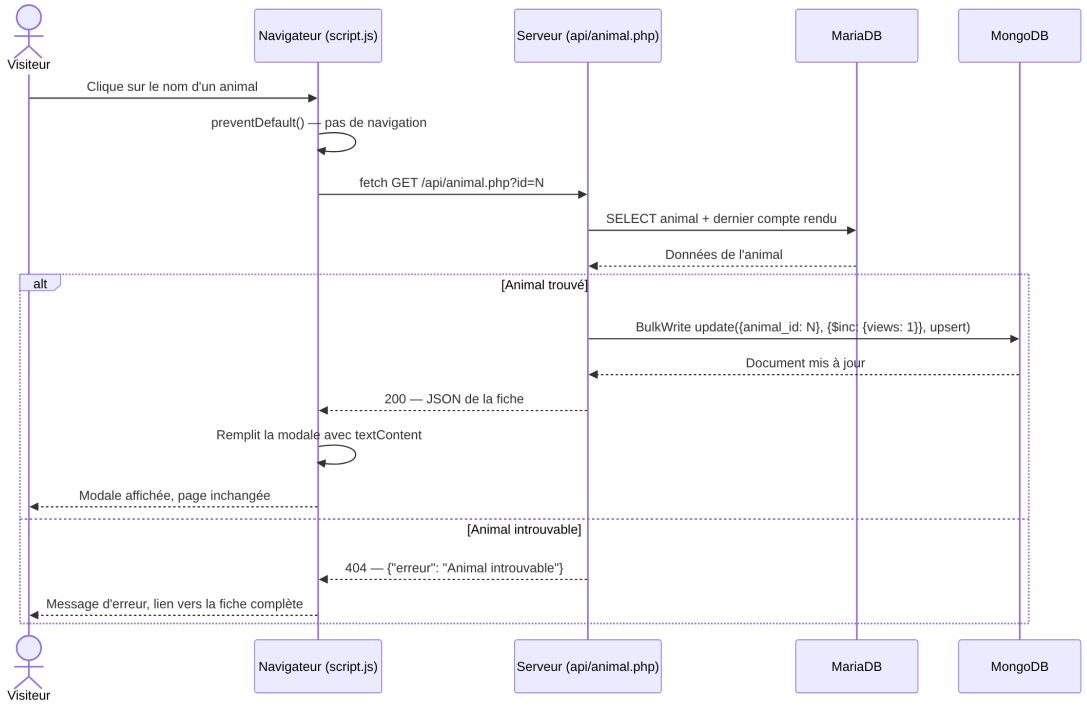
Connexion avec contrôle de rôle



Régénération de l'identifiant de session. `session_regenerate_id(true)` remplace l'identifiant de session au moment précis où l'utilisateur gagne des droits, et supprime l'ancien. Sans cet appel, un attaquant qui aurait imposé à sa victime un identifiant connu de lui, avant la connexion, le retrouverait valide et privilégié après. C'est l'attaque par fixation de session. La régénération est la contre-mesure recommandée par l'OWASP.

Un message d'erreur générique. Les trois cas d'échec (compte inexistant, mot de passe incorrect, compte désactivé) renvoient le même message. Le diagramme les regroupe en deux branches : « compte inexistant » et « mot de passe incorrect » tombent tous deux dans « Identifiants invalides ». C'est délibéré : un message différent pour chaque cas indiquerait à un attaquant quelles adresses existent réellement dans la base.

Consultation d'une fiche animal en requête asynchrone (compteur MongoDB)



Un parcours qui fonctionne aussi sans JavaScript. Le lien `animal.php?id=N` reste actif. Sans JavaScript, il sert la même information sous forme de page complète. Le diagramme décrit le parcours amélioré, pas le seul parcours possible.

Un compteur incrémenté seulement pour un animal existant. L'incrément figure dans la branche « animal trouvé ». C'est pourquoi `incrementer_consultation()` est appelée après le contrôle d'existence : une consultation n'est comptée que pour un animal qui existe réellement.

6. Documentation du déploiement

L'application est déployée à l'adresse https://lesstc.alwaysdata.net/.

Choix de l'hébergeur

Les deux plateformes envisagées au départ (proposées dans l'énoncé) ne convenaient plus au moment de déployer car elles sont devenues payantes.

J'ai retenu alwaysdata, sur son plan gratuit à 100 Mo : PHP, MariaDB, SSH, SFTP et certificat Let's Encrypt inclus, avec une infrastructure hébergée en France.

Ce choix est cohérent avec l'exigence d'écoresponsabilité du cahier des charges. Le principal levier sur l'empreinte carbone d'un hébergement est le mix électrique du centre de données. alwaysdata publie une politique environnementale qui suit la

hiérarchie sobriété → efficacité → électricité bas-carbone → compensation. La compensation carbone vient en dernier recours, et non comme argument principal.

Préparation avant déploiement

Je développe en local sous Windows (XAMPP), alors que le serveur d'alwaysdata tourne sous Linux, comme la plupart des hébergeurs web. Le passage d'un système à l'autre fait remonter des détails invisibles en local. Le principal : Linux distingue les majuscules des minuscules dans les noms de fichiers, contrairement à Windows. Un lien vers `services.php`, alors que le fichier s'appelle `Services.php`, fonctionne sous Windows mais renvoie une erreur 404 sous Linux. C'est la cause la plus fréquente d'un déploiement qui échoue sans raison apparente.

J'ai donc vérifié :

- la casse de mes 11 fichiers PHP face aux liens du code (concordance exacte) ;
- les 20 références aux images face aux fichiers réels ;
- l'absence de chemin absolu ou de `localhost` codé en dur (tout passe par `config.php` ou `__DIR__`) ;
- la compatibilité avec PHP 8.4.

Ces vérifications n'ont révélé aucun écart : le code était portable en l'état. Le dépôt pesait 6,6 Mo pour un quota de 100 Mo.

Trois modifications ont néanmoins été faites en préparation, chacune sur sa propre branche Git :

1. **Des scripts SQL portables.** J'ai retiré `CREATE DATABASE` et `USE` : chez un hébergeur mutualisé, la base existe déjà et son nom est imposé.
2. **Trois anciennes migrations supprimées.** Elles n'étaient plus exécutables sur une installation neuve.
3. **Un cookie de session durci**, avec les attributs `SameSite=Lax`, `HttpOnly`, et `Secure` conditionnel.

Étapes du déploiement

1. Création du compte alwaysdata et du sous-domaine.
2. Création de la base `lesstc_arcadia` en `utf8mb4`.
3. Ouverture de l'accès SSH.
4. Synchronisation de `main` depuis `develop`.

5. Récupération du code sur le serveur par `git clone`.
6. Création de `config/config.php` directement sur le serveur, en `chmod 600`.
7. Validation de la connexion PDO, avant tout import de données.
8. Import des deux scripts SQL : 11 tables, 3 habitats, 12 animaux, 3 services, 3 avis.
9. Configuration du site : racine, version de PHP, redirection HTTPS forcée.
10. Création des trois comptes du back-office avec `sql/03_create_user.php`.
11. Première ouverture du site en conditions réelles.
12. Tests post-déploiement, page par page et code HTTP par code HTTP.
13. Correctif de sécurité : ajout du fichier `.htaccess` à la racine.
14. Compilation de l'extension MongoDB pour PHP 8.4.
15. Création du cluster MongoDB Atlas (offre gratuite M0) et branchement à l'application.

Difficultés rencontrées

Des fichiers sensibles exposés. C'est la difficulté la plus importante pour la sécurité. J'ai testé systématiquement ce qui était accessible publiquement, y compris des adresses vers lesquelles le site ne fait aucun lien. Deux fichiers répondaient avec un contenu lisible :

- `.git/config`, qui permettait de reconstituer l'historique du dépôt ;
- `sql/01_schema.sql`, qui exposait la structure complète de la base.

`config/config.php` répondait aussi avec un code 200, mais 0 octet : Apache l'exécute via PHP au lieu de le servir en clair. Rien n'y fuyait, mais cette protection reposait sur la configuration du serveur, pas sur le code.

J'ai ajouté à la racine un fichier `.htaccess` qui bloque toute requête directe vers les dossiers `.git`, `config`, `sql` et `courriels`. J'ai vérifié, en local puis en production, que ces chemins répondent bien 403, tandis que l'accueil et les pages du site répondent toujours 200.

Une connexion SSH par clé refusée. Le serveur refusait ma clé, alors que la clé publique correspondait bien à celle enregistrée. Deux causes possibles, non départagées :

- le dossier personnel appartenait à `root` et était inscriptible par le groupe, ce qu'OpenSSH refuse par principe (le mécanisme s'appelle `StrictModes`) ;

- le champ « répertoire HOME » de l'utilisateur pointait vers un dossier inexistant.

J'ai corrigé le second point, mais faute de temps je n'ai pas retesté la connexion par clé. Je suis donc passée par une connexion par mot de passe, en sachant que la cause réelle n'est peut-être traitée qu'en partie.

MongoDB absent de l'offre gratuite. Il manquait deux éléments distincts. D'abord le pilote : l'extension qui permet à PHP de communiquer avec MongoDB. Ensuite le serveur : l'endroit où sont stockés les documents. J'ai compilé le pilote avec un script fourni par l'hébergeur. Pour le serveur, j'ai créé un cluster gratuit sur MongoDB Atlas, puis j'ai ajouté son adresse dans `config.php`.

Le code de l'application n'a pas changé d'une ligne. Toutes les adresses et tous les mots de passe des bases sont rangés dans `config.php`, et nulle part ailleurs dans le code. Passer d'un MongoDB local à un cluster distant a donc demandé de modifier une seule valeur dans ce fichier.

Validation finale

PHP en ligne de commande et PHP servi par Apache ne partagent pas forcément la même configuration : une extension peut être active d'un côté et absente de l'autre. J'ai donc testé le compteur MongoDB en deux temps. D'abord en ligne de commande. Puis en conditions réelles : j'ai consulté six fiches animaux depuis l'extérieur et vérifié que les compteurs correspondaient exactement au nombre de clics. C'est ce second test qui compte : il prouve que l'US 11 fonctionne en ligne, devant le jury, et pas seulement en local.

L'état vérifié en production :

- l'accueil et les pages habitats, services, contact et connexion répondent 200 ;
- un espace professionnel sans connexion redirige (302) vers la page de connexion ;
- un identifiant d'animal ou d'habitat inexistant dans l'URL renvoie 404 ;
- le site force la redirection de HTTP vers HTTPS (301) ;
- le cookie de session porte les attributs `Secure`, `HttpOnly` et `SameSite=Lax` ;
- les chemins sensibles (`.git`, `config`, `sql`, `courriels`) répondent 403 ;
- le compteur MongoDB fonctionne depuis le web.

L'attribut `Secure` du cookie est inactif en local, puisque XAMPP sert le site en HTTP. Il s'est activé automatiquement au déploiement, grâce à une condition posée dans le

code (`!empty($_SERVER['HTTPS'])`). Le même code s'adapte donc au protocole réel, sans configuration distincte entre développement et production.

Limites assumées

- **SSH par mot de passe plutôt que par clé.** Des deux causes identifiées, l'une a été corrigée. L'autre, les permissions du dossier personnel, n'est pas modifiable sur un hébergement mutualisé. La connexion par clé n'a pas été retestée après cette correction partielle : c'est une piste ouverte plutôt qu'un échec établi.
- **Extension MongoDB liée à la version de PHP.** Un passage à une version majeure supérieure demanderait de la recompiler.
- **Liste d'IP autorisées largement ouverte sur Atlas.** L'adresse IP sortante d'un hébergement mutualisé n'est pas stable. La protection repose alors sur le mot de passe de l'utilisateur applicatif.
- **Courriels journalisés en fichier.** Ils ne sont pas réellement envoyés, faute d'envoi SMTP configuré en production.
- **Pas de jeton anti-CSRF.** Le risque est mitigé par les attributs `SameSite=Lax`, `HttpOnly` et `Secure` du cookie de session.
- **Images ajoutées par chemin.** Un véritable téléversement exigerait une validation stricte du type réel de fichier.

Procédures et méthode

Le déploiement initial est un `git clone` du dépôt public sur le serveur, suivi de la création de `config/config.php` directement là où le code tourne. Une mise à jour tient en une commande côté serveur, après avoir fusionné les changements dans `main` et poussé sur GitHub :

```
cd /home/lesstc/www/arcadia && git pull
```

Cette organisation repose sur trois règles :

1. **Tout passe par Git.** Le déploiement est un `git clone`, la mise à jour un `git pull`. Il n'y a aucun transfert manuel de fichiers, donc aucun risque de divergence entre le dépôt et le serveur.
2. **La configuration est sortie du code.** `config.php` est exclu du dépôt et créé directement sur le serveur. Aucun identifiant de production n'a jamais transité

par GitHub, et changer de base ou d'hébergeur ne demande aucune modification du code applicatif.

3. **Le déploiement se fait toujours depuis** `main`, jamais depuis `develop`. La branche principale reflète ainsi en permanence ce qui tourne en production.

Trois principes m'ont débloquée dans la plupart des situations :

1. **Isoler les variables.** J'ai testé la connexion à la base seule, avant tout import, pour séparer une erreur de configuration d'une erreur de données.
2. **Lire le message d'erreur en entier**, pas seulement sa première ligne. `Access denied ... (using password: YES)` indique déjà que trois paramètres sur quatre sont corrects.
3. **Changer de canal de diagnostic quand on tourne en rond.** Après plusieurs échecs sur une saisie de mot de passe invisible en SSH, je suis passée par phpMyAdmin. Son formulaire web affiche un message d'erreur explicite.